

INTRODUCTION TO PROGRAMMING (CSE141)

Lab Examination – Set B

Max Marks: 40 | Time Allowed: 2 Hours

Submission folder: Name_ERP (e.g., Ali_20261234). Save all four specified .cpp files in this folder.

Exercise 1: Parking Fee Machine

[10 marks]

Read the number of vehicles, then read each vehicle's type code (1, 2, or 3) and hours parked (an integer) into static arrays. Use a **switch** on the type code for the rate:

- Type 1 car: Rs. 50/hour, Type 2 van: Rs. 80/hour, Type 3 truck: Rs. 120/hour

If a vehicle parked more than 5 hours, give it a 10 percent discount on its total. Display each fee and the total collected. Money is **double**.

Sample Input

```

Number of vehicles: 2
1 6
3 2
  
```

Sample Output

```

Vehicle 1: Rs. 270.00
Vehicle 2: Rs. 240.00
Total: Rs. 510.00
  
```

Marking rubric:

Rubric	Points
Reads counts, types, and hours into arrays correctly	2
Correct rate selection using switch	2
Correct fee and discount logic	3
Correct running total	2
Clean output and readable code	1

Submission: fee.cpp

Exercise 2. Numerical Badge Inspection 10 marks

A proctor enters n integers (1..100) in sequence. A **NEGATIVE** integer represents a damaged badge: skip it without checking. **ZERO** stops inspection immediately; ignore all remaining values even if they exist on input. For each **POSITIVE** integer, examine its decimal digits from **RIGHT** to **LEFT**. Add digits in positions 1,3,5,... into one total and digits in positions 2,4,6,... into another (the rightmost digit is position 1). Accept the badge if the **ABSOLUTE DIFFERENCE** of these totals is divisible by 3.

Print how many positive badges were checked before stopping, how many were accepted, and the **LARGEST** accepted number. If no positive badge was accepted, print -1. Do not turn integers into strings and do not use functions.

Submission file: badge.cpp

Sample Input — Keyboard input

```

8
-5 123 333 741 660 0 9090 808
  
```

Sample Output — Console output

```

Checked: 4
Accepted: 2
Largest accepted: 660
  
```

Marking rubric

Assessment criterion	Marks
Break on zero and continue for negatives	2
Digit extraction and alternating positions	4
Divisibility / absolute difference	2
Counts and largest accepted (including none)	2
TOTAL	10

Exercise 3: The Conveyor Belt (Case Study)

[10 marks]

A factory conveyor holds items in a grid of $m \times n$ integers. At the start of a shift the grid is loaded as shown. After the belt runs for a while, the arrangement changes to the second grid below. You must read the starting grid and an integer k from the user, work out **what the belt is doing**, and reproduce the final arrangement.

Compare the before and after and decide for yourself whether the belt shifts rows, shifts columns, rotates, or transposes, then write the code. The value shown corresponds to $k = 1$.

Starting grid

```
10 20 30 40
50 60 70 80
```

After running ($k = 1$)

```
40 10 20 30
80 50 60 70
```

Your program reads m , n , k , and the starting grid, then prints the final grid.

Sample Input

```
2 4 1
10 20 30 40
50 60 70 80
```

Sample Output

```
40 10 20 30
80 50 60 70
```

Marking rubric:

Rubric	Points
Reads m , n , k and the grid into a 2D array correctly	2
Identifies the correct operation (row shift right by k) from the example	3
Correct wrap-around for any k , using nested loops	4
Correct grid output	1

Submission: belt.cpp

Exercise 4: Temperature File Study [10 marks]

A file temps.txt holds an unknown number of integer readings, one or more per line (at most 100).

- Read all readings into a static array.
- Compute the average as a double.
- Count how many readings are strictly above the average.
- Arrange the readings from highest to lowest using **selection sort** (one step of the task), then write the ordered list and the average to result.txt.

If the file cannot be opened, print an error message and stop.

Sample Input

```
temps.txt:
30 42 28
35 41
```

Sample Output

```
Average: 35.20
Above average: 2
(result.txt written)
```

Marking rubric:

Rubric	Points
Opens file with error check; reads unknown count into array	2
Correct average and above-average count	3
Correct selection sort (descending) as a sub-step	3
Writes ordered list and average to output file	2

Submission: tempstudy.cpp